

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
"НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ"

КАФЕДРА ТЕОРЕТИЧЕСКОЙ И ПРИКЛАДНОЙ ИНФОРМАТИКИ

Лабораторная работа № 4
по дисциплине "Проектирование интерфейсов и системный анализ"

Факультет: ФПМИ

Группа: ПМИ-32

Студенты: Весёлый Д., Ворончук И., Лыкова М.

Бригада №2

Преподаватели: Кутузова И. А.

НОВОСИБИРСК 2026

Цель: формирование навыков создания прототипа интерфейса windows-приложения в соответствии с принципами проектирования пользовательского интерфейса.

Вариант	Тема
2	Книжный каталог

Принцип структуры: Интерфейс должен иметь чёткую логическую организацию. Связанные элементы группируются вместе, несвязанные - разделяются. Пользователь должен сразу понимать, где искать нужный элемент.

Пример: Логическая группировка на главной форме. Главная форма (MainForm) разбита на три функциональные зоны, расположенные по принципу “сверху вниз”:

Панель поиска (GroupBox "Поиск книг")	фильтрация
Таблица книг (DataGridView)	данные
[Подробнее] [Забронировать] [Войти]	действия
Строка состояния: "Всего: 6 Дост.: 3"	информация

Панель поиска дополнительно объединена в GroupBox с заголовком "Поиск книг", что визуально отделяет её от остального содержимого. Поисковые поля сгруппированы логически: название и автор - на одной строке, жанр - рядом, год - снизу.

Принцип простоты: Интерфейс должен быть максимально простым и понятным. Пользователь не должен угадывать назначение элементов. Текст кнопок и подписей должен однозначно описывать действие.

Пример: Все кнопки в приложении используют глагольные формулировки, точно описывающие действие.

```
btnDetails.Text = "Подробнее";    // не "Детали" и не "Info"  
btnReserve.Text = "Забронировать"; // не "Reserve" и не "Бронь"  
btnAddBook.Text = "Добавить книгу"; // не "Add" и не "Новая"  
btnWriteOff.Text = "Списание";    // не "Удалить" (это другое)  
btnLogin.Text   = "Войти";        // не "Авторизация"  
btnLogout.Text  = "Выйти";        // не "Деавторизация"
```

Подтверждающие диалоги содержат конкретную информацию о действии, а при поиске ошибки валидации сообщают в каком именно поле проблема.

Принцип видимости: пользователь должен видеть только те действия, которые ему доступны в данный момент. Недоступные функции не отвлекают внимание. Интерфейс адаптируется под текущую роль и состояние.

Пример: Скрытие кнопок по роли пользователя. Метод `ApplyRolePermissions()` вызывается при каждой смене состояния (вход, выход, возврат из дочерней формы) и управляет видимостью элементов. Роли определены в модели [User.cs](#). Кнопка "Подробнее" и "Забронировать" отключаются, когда в таблице не выбрана строка.

Принцип обратной связи: Каждое действие пользователя должно получать немедленный и понятный отклик. Пользователь должен понимать, что произошло: успех, ошибка или нейтральный результат.

Пример 1: Цветовая индикация доступности книги. В форме деталей книги (`BookDetailsForm`) доступность показывается цветом и состоянием кнопки одновременно.

Пример 2: После успешного бронирования форма обновляет отображение и показывает подтверждение.

Пример 3: Строка состояния в нижней части формы отображает актуальную статистику. При поиске она заменяется на количество найденных результатов.

Принцип толерантности: Интерфейс должен прощать ошибки. Действия должны быть обратимыми или требовать подтверждения. Вводимые данные должны валидироваться до их обработки. Сообщения об ошибках должны подсказывать, как исправить проблему.

Пример 1: Валидация ISBN с контрольной суммой. Форма добавления книги проверяет ISBN по алгоритмам ISBN-10 и ISBN-13, включая вычисление контрольной цифры. Метод `IsValidISBN()` вычисляет контрольную сумму по модулю 11 (ISBN-10) или 10 (ISBN-13).

Пример 2: Перед списанием книги пользователь видит диалог с точным количеством и причиной. Если у книги есть активные брони - система не списывает её молча, а предупреждает и спрашивает подтверждение.

Принцип единообразия: Все элементы интерфейса должны выглядеть и вести себя одинаково. Одинаковые действия вызывают одинаковый результат. Единый визуальный стиль, шрифты, цвета и паттерны взаимодействия.

Пример 1: Единая тема через `Theme.Apply()`. Все формы приложения вызывают `Theme.Apply(this)` в конструкторе, что гарантирует единообразный внешний вид. Класс `Theme` определяет единую палитру для всех элементов.

Пример 2: Все переходы между формами используют один и тот же паттерн - скрывание родительской формы, открытие дочерней через `ShowDialog()`, возврат с обновлением.

Пример 3: Все формы используют одинаковый подход к выводу ошибок - `MessageBox` с иконкой `Warning` и фокусировкой на проблемном поле.

Заключение: Все шесть принципов проектирования интерфейса последовательно применены в приложении `BookCatalogWinForms`. Структура форм следует логической схеме "поиск - данные - действия - статус". Простота обеспечивается понятными текстами кнопок и диалогов. Видимость реализована через ролевую модель - каждый пользователь видит только доступные ему функции. Обратная связь предоставляется

через цветовую индикацию, обновление данных в реальном времени и информативную строку состояния. Толерантность достигается валидацией ввода (ISBN, год, количество), подтверждающими диалогами и возможностью восстановления списанных книг. Единообразие обеспечивается централизованным классом Theme и унифицированным паттерном навигации между формами.